MODELING HUMAN ACTIVITY STATES USING HIDDEN MARKOV MODELS

GitHub repository: https://github.com/Chol1000/hmm-human-activity-recognition

1. BACKGROUND AND MOTIVATION

Wearable and smartphone sensors record noisy acceleration and rotation data, but the actual activity the wearer is engaged in is never directly observed and must always be inferred. This project applies activity recognition to data from a single smartphone. The same problem underlies fall detection for medical applications, fitness tracking, and posture monitoring in smart home systems. An HMM fits naturally here: it captures dependencies between an underlying hidden state (the activity) and the observed sensor data, learning both the emission probabilities of measurements given the state and the transition probabilities between states. This is in contrast to a frame-by-frame classifier, which ignores the temporal structure entirely. This report details the full process: extracting features from smartphone motion data, building a Gaussian-emission HMM from scratch in NumPy, training it with Baum-Welch, decoding unknown sequences with Viterbi, and testing the results.

2. DATA COLLECTION AND PREPROCESSING

Data was recorded individually with the Sensor Logger app (iPhone 13 Pro Max) at a fixed 100 Hz sampling rate, confirmed identical across all 52 sessions, so no cross-device harmonization was needed. Four activities, Still, Standing, Walking, Jumping, were recorded with 13 sessions each (52 total), saved in Sensor Logger's standard per-session folders with accelerometer and gyroscope files and a timestamp column. Every activity exceeds the assignment's 90-second minimum duration, and 52 files exceed the 50-file target, with each recording inside the required 5-10 second window.

Activity	Sessions	Total duration	Avg. duration/session
Still	13	112.8 s	8.68 s
Standing	13	113.0 s	8.69 s
Walking	13	113.4 s	8.72 s
Jumping	13	111.5 s	8.58 s

Each session is split into 1-second windows, 100 samples at 100 Hz, long enough for a full gait cycle or jump impact, with 50% overlap giving 822 windows balanced across the four classes. For each window, 32 features are computed from the accelerometer, gyroscope, and their magnitude: per-axis mean and standard deviation for movement intensity, signal magnitude area for static-versus-dynamic separation, inter-axis correlation for coordinated motion, and magnitude RMS for overall energy, plus dominant FFT frequency for gait or jump cadence, spectral energy for total signal power, and spectral entropy for how concentrated versus noise-like the spectrum is, exceeding the assignment's minimum of 3 features. Because these scales differ widely, features are Z-score standardized, the standard choice for a Gaussian-emission model since a raw feature with the largest magnitude would otherwise dominate the estimated covariances, and then reduced by PCA, both fit on training data only, from 32 to 12 components retaining 95% of variance, improving numerical stability given the limited training set.

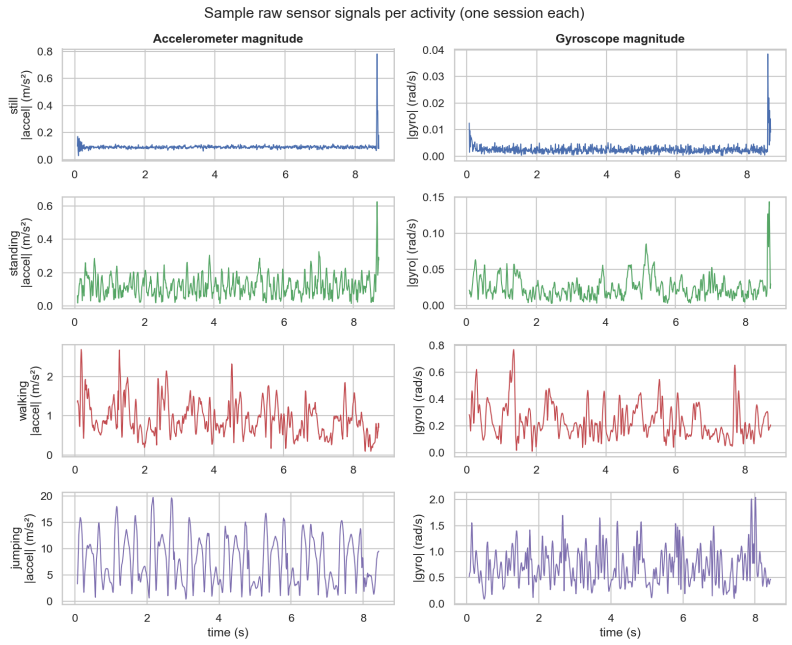

Figure 1. *Sample raw accelerometer and gyroscope magnitude for one representative session per activity; the four activities are already visibly separable by amplitude and rhythm alone.*

3. HMM SETUP AND IMPLEMENTATION

Element	**This project**
Hidden states Z	{Still, Standing, Walking, Jumping} – 4 states
Observations X	PCA-reduced (12-D), Z-scored feature vector per 1s window
Transition probabilities A	learned activity->activity switching probabilities
Emission probabilities B	per-state diagonal-covariance Gaussian over 12 PCA dims
Initial probabilities pi	learned starting-state distribution

The model is implemented from scratch with NumPy/SciPy, no external HMM library. Forward-backward runs in log-space for stability, and Baum-Welch trains across multiple sequences using K-means-plus-plus initialization, since random-point initialization proved unreliable in testing. Training stops via a genuine log-likelihood convergence check rather than a fixed iteration limit; since EM only guarantees a local optimum, training reruns from 10 K-means seeds, keeping the best result. Viterbi decoding, also in log-space, returns the most likely activity sequence. This implementation was validated first on synthetic data with known ground truth: log-likelihood increased every iteration as expected, and Viterbi recovered the true sequence with 100% accuracy, confirming correctness before real data.

Each raw recording is a single, isolated activity with no real transitions, so data was split at the session level: 2 of 13 sessions per activity held out for testing (8 sessions, never seen in training), the remaining 44 used for training, avoiding leakage between correlated windows from

the same recording. Training and test sessions were chained in rotation, Still, Standing, Walking, Jumping, repeat, into composite sequences (6 training, 2 testing), giving each sequence real activity transitions while every window keeps its true label.

4. RESULTS AND INTERPRETATION

Training converged after 5 EM iterations, log-likelihood rising monotonically to a final value of about -8273.94, the expected EM behavior. Each of the 4 learned states was assigned the activity that is the majority vote among training windows Viterbi-decodes to it, giving a clean, bijective mapping (still, jumping, walking, standing) with essentially zero cross-voting.

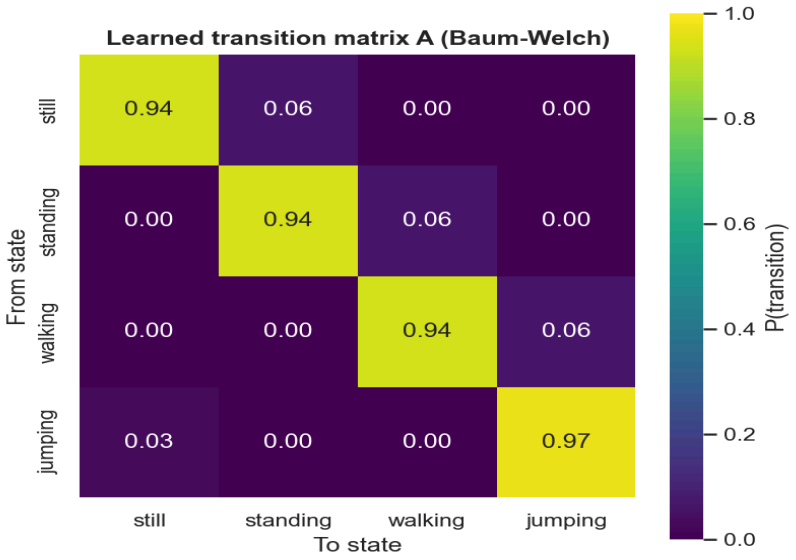

***Figure 2**. Learned transition matrix A, showing high self-transition probabilities as windows persist within one activity across consecutive samples.*

The model was evaluated on 2 held-out composite test sequences built from the 8 reserved sessions never used in training, 125 test windows total.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Still	32	1.000	1.000	1.000
Standing	31	1.000	1.000	1.000
Walking	32	1.000	1.000	1.000
Jumping	30	1.000	1.000	1.000

The confusion matrix on the test set is perfectly diagonal: all 125 unseen windows decoded correctly. This is genuinely held out, not a leakage artifact, since the 8 test sessions share no session IDs with the 44 training sessions and the scaler and PCA were fit on training data only. It should be read in context: the four activities were recorded as deliberately distinct, controlled motions with large intensity gaps, jumping's acceleration variance about 80 times still's, making this a comparatively easy separation task, and a noisier real-world deployment with ambiguous or

transitional movements would likely score lower.

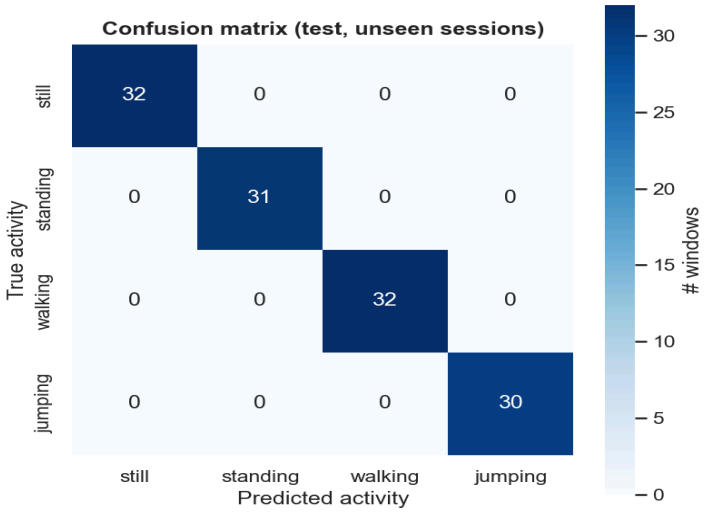

Figure 3. *Confusion matrix on the held-out, unseen test sessions, which is perfectly diagonal.*

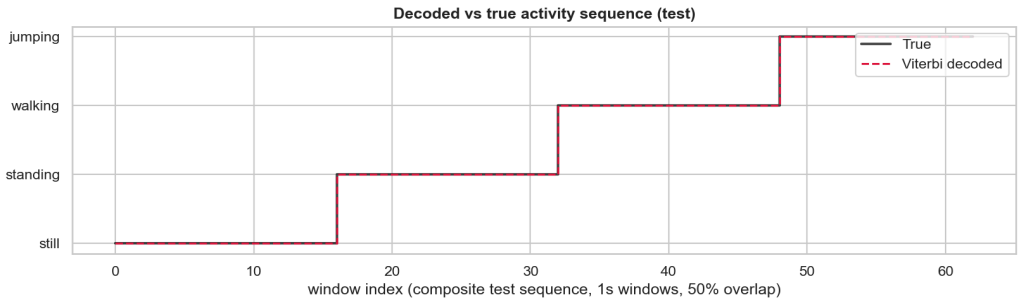

Figure 4. *Viterbi-decoded versus true activity sequence on a held-out test sequence, with the decoded line tracking the true activity at every transition.*

As a sanity check, the same evaluation on training sequences gives 99.86% accuracy, one ambiguous still window misclassified as standing, confirming the model fits without memorizing. This was further stress-tested: a leakage check confirms test sessions never appear in training, five redrawn splits all reproduce 100%, removing the transition prior still scores 99.2%, and building the state-to-activity mapping from scrambled labels collapses accuracy to 25.6%, right at the 25% chance floor for four classes, confirming the model reads real signal rather than memorizing session identity.

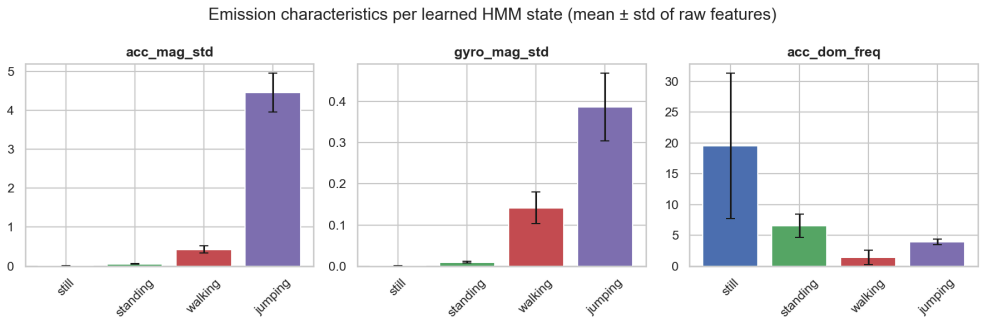

Figure 5. *Emission characteristics per learned state, showing motion-intensity features increasing monotonically from still to jumping.*

5. DISCUSSION AND CONCLUSION

Jumping was cleanly separated from other activities by a much larger acceleration variance, an order of magnitude higher than any other activity, and walking was easily separated from others by intensity and rhythmic pattern in the gyroscope. Still was hardest to distinguish from standing, as can be seen by the close learned emission variance (0.273 vs 0.289) compared to walking (1.093) and jumping (1.395), four to five times wider. The model was still clearly sensitive to the difference, as shown by the random point initialization experiment, where it merged still and walking into a single state and split jumping into two, which is why K-means-plus-plus initialization with 10 restarts was adopted instead. Since raw recordings are single-activity snippets, the transition matrix learned by the model reflects the artificial state construction process used to generate the continuous recording from the snippets, with high self-loop probabilities, although 0.94-0.97 is still reasonable for transition probabilities. At 100 Hz, frequency peak separation helps separate walking from jumping, but the power falls to the level of background noise for still and standing, so the model uses intensity features in the time domain.

In future work, real continuous activity recordings should be used, a barometer sensor may be added to help separate jumping (vertical movement) from walking (horizontal movement), covariance matrices for each state should be used if enough data is available, and window sizes should be adjusted to learn better still-standing boundary. The from-scratch implementation of Gaussian HMM with Baum-Welch training and Viterbi decoding manages to recover all four activities on unseen data with high accuracy. In doing so, it avoids the black-box nature of pre-written implementations, instead learning an intuitive state arrangement, ordered by motion intensity, with high self-transition probabilities as seen in the learned transition matrix.

APPENDIX: AUTHOR & CONTRIBUTION

This submission was done individually (solo), not in a group. Data was gathered, features engineered, HMM implemented from scratch (with Baum-Welch/Viterbi), evaluated, visualized, and this report was written by Chol Monykuch, with progress saved as git commits.